



1. Problem Title & Link

Title: LeetCode 643 : Maximum Average Subarray I

Link: [LeetCode 643 - Maximum Average Subarray I](#)

2. Problem Statement (Short Summary)

You are given:

nums[]

k

Find the contiguous subarray of size k that has the maximum average value.

Return the maximum average.

In simple terms:

- Choose exactly k consecutive elements
- Find their average
- Return the largest possible average

3. Examples (Input → Output)

Example 1

Input:

nums=[1,12,-5,-6,50,3]

k=4

Output:

12.75

Explanation:

Subarray:

[12,-5,-6,50]

Sum:

51

Average:

$51/4=12.75$

Example 2

Input:

nums=[5]

k=1

Output:



5.0

Example 3

Input:

nums=[0,4,0,3,2]

k=1

Output:

4.0

4. Constraints

$1 \leq k \leq \text{nums.length} \leq 10^5$

$-10^4 \leq \text{nums}[i] \leq 10^4$

Special restrictions:

- Subarray size is fixed
- Need efficient solution

5. Core Concept (Pattern / Topic)

Sliding Window

6. Thought Process (Step-by-Step Explanation)

Brute Force

For every subarray of size k:

1. Compute sum
2. Compute average
3. Track maximum

Time:

$O(n*k)$

Too slow for large arrays.

Optimized Thinking

Observation:

When moving from one window to the next:

Remove left element

Add right element

Instead of recalculating full sum every time.



Algorithm:

1. Compute first window sum
2. Slide window:
 - subtract outgoing element
 - add incoming element
3. Track maximum sum
4. Return:

maxSum/k

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

nums=[1,12,-5,-6,50,3]

k=4

Window:

[1,12,-5,-6]

sum=2

Slide:

Remove 1

Add 50

[12,-5,-6,50]

sum=51

Maximum average:

$51/4=12.75$

8. Pseudocode (Language Independent)

windowSum=sum(first k elements)

maxSum=windowSum

for i from k to n-1:

 windowSum += nums[i]



```
windowSum -= nums[i-k]
```

```
maxSum=max(maxSum>windowSum)
```

```
return maxSum/k
```

9. Code Implementation

Python

class Solution:

```
def findMaxAverage(
```

```
    self,
```

```
    nums,
```

```
    k):
```

```
    windowSum=sum(nums[:k])
```

```
    maxSum=windowSum
```

```
    for i in range(
```

```
        k,
```

```
        len(nums)
```

```
    ):

```

```
        windowSum += nums[i]
```

```
        windowSum -= nums[i-k]
```

```
        maxSum=max(
```

```
            maxSum,
```

```
            windowSum
```

```
        )
```

```
    return maxSum/k
```

Java



```
class Solution {  
  
    public double findMaxAverage(  
        int[] nums,  
        int k) {  
  
        int windowSum=0;  
  
        for(int i=0;i<k;i++){  
            windowSum+=nums[i];  
  
        int maxSum=windowSum;  
  
        for(int i=k;  
            i<nums.length;  
            i++){  
  
            windowSum+=nums[i];  
  
            windowSum-=  
                nums[i-k];  
  
            maxSum=  
                Math.max(  
                    maxSum,  
                    windowSum  
                );  
        }  
  
        return  
            (double)maxSum/k;  
    }  
}
```

10. Time & Space Complexity

Time Complexity:

$O(n)$



Reason:

Each element enters and exits the window once.

Space Complexity:

$O(1)$

Reason:

Only variables are used.

11. Common Mistakes / Edge Cases

1. Recalculating sum for every window

Wrong:

Nested loop

2. Forgetting floating-point division

Wrong:

$51/4 = 12$

Correct:

12.75

3. Incorrect window update

Correct order:

Add new element

Remove old element

4. Missing single element case

nums=[5]

k=1

12. Detailed Dry Run (Step-by-Step Table)

Input:

nums=[1,12,-5,-6,50,3]

k=4

Window	Window Sum	Max Sum
--------	------------	---------



[1,12,-5,-6]	2	2
[12,-5,-6,50]	51	51
[-5,-6,50,3]	42	51

Final Average:

$$51/4=12.75$$

13. Common Use Cases (Real-Life / Interview)

Stock market trend analysis

Sensor data averaging

Network traffic monitoring

Performance analytics

14. Common Traps (Important!)

1. Using nested loops
2. Integer division mistakes
3. Wrong window size updates
4. Forgetting first window initialization

15. Builds To (Related LeetCode Problems)

Progression:

- LC 643 → Maximum Average Subarray I
- LC 209 → Minimum Size Subarray Sum
- LC 3 → Longest Substring Without Repeating Characters
- LC 424 → Longest Repeating Character Replacement
- LC 76 → Minimum Window Substring

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n*k)$	$O(1)$	Repeated sum calculation
Prefix Sum	$O(n)$	$O(n)$	Extra memory
Sliding Window	$O(n)$	$O(1)$	Optimal

17. Why This Solution Works (Short Intuition)



The window size remains fixed at k . Instead of recomputing sums repeatedly, the sliding window updates the sum efficiently by removing one element and adding another.

18. Variations / Follow-Up Questions

1. Return subarray indices
2. Find minimum average subarray
3. Variable-sized window maximum average
4. Maximum sum subarray of size k
5. Streaming average calculation
- 6.